reliably. Embedding authorization logic in application code causes duplication and slows down updates. Instead, the team wants policies that can be expressed in their own language, externalized from applications but still precise enough to support real-time decisions.

Of course, policies don't exist in a vacuum. Deciding who writes them, who reviews them, and how conflicts are resolved is just as important as the policies themselves. That aspect, known as *policy governance*, is a key part of making authorization sustainable, and we will revisit it in Chapter 14.

This chapter presents three perspectives on authorization policy. The first is the *what:* principles for writing clear, consistent policies that enforce least privilege and remain stable as organizations grow. The second is the *how:* treating policies as code so they can be reviewed, versioned, and deployed like any other software asset. The third is *assurance:* testing and analysis that confirm policies behave as intended and stay trustworthy over time. Together, these perspectives ensure that authorization logic isn't just theoretical but can be managed, verified, and adapted in practice.

10.1 Principles of effective authorization policies

As ACME's identity team expanded its use of dynamic authorization, they realized that writing policies is more than just encoding rules in Cedar. The real challenge is making policies effective—clear enough for policy authors and reviewers to understand, stable enough to withstand organizational changes, and precise enough to enforce least privilege without getting in the way of business goals.

The following sections outline several principles that ACME found to guide its work.

10.1.1 Clarity and consistency

Policies should be clear and predictable. A policy that's vague or that conflicts with others is a liability. At ACME, the Customer Collaboration platform once had two policies that seemed to conflict: one granting managers broad access and another restricting access based on project